数据结构与算法 A

2021 年期中 · 试卷

整理说明：来源为 20 张课堂投影照片，题目与讲解混排。本卷按可见题号复原，移除答案。第二部分第 3、4 题未见原图；算法填空第 1 题代码有缺页，采用注明的结构性复原。原图未能确认的分值不补写。

姓名：　　学号：　　日期：

一、选择填空

1．以下算法的时间复杂度是（　）。

```
int m = 1, i, j;
for (i = 1; i <= n; i++)
    for (j = 1; j <= i; j *= 2)
        m *= 2;
```

A. $\Theta(\log_2 n)$　B. $\Theta(n)$　C. $\Theta(n \log_2 n)$　D. $\Theta(n^2)$

2．在（　）中，即使丢失了头结点，只要指出表中任何一个结点的指针，也可以访问到该结点的前驱结点。（可多选）

A. 线性单链表　B. 双向链表　C. 线性链表　D. 循环链表

3．采用教材上将中缀表达式转换为后缀表达式的过程中，栈中不可能出现的状态是（　）。（左侧为栈底，可多选）

A. +/+　B. +(+　C. (+)　D. +*(+

4．双端队列的插入和删除操作限制在两端进行。若将 n 个互不相同的元素依次插入双端队列中（仅插入，不删除），可得到多少种不同排列？（　）

A. 2　B. $2n$　C. 2^{n-1}　D. 2^n

5．顺序存储的循环队列 Q[1:40]，数组下标为 1 到 40，front 指向队头元素，rear 指向队尾元素的下一位置。初始 front = rear = 40，经过一系列入队与退队后 front = 20、rear = 15。寻找队列中最小元素，最坏情况下需要比较多少次？

6．二叉树后序遍历为 {1,3,2,6,5,7,4}，中序遍历为 {1,2,3,4,5,6,7}，下列哪句是错的？（　）

A. 2 是 1 和 3 的父结点　B. 7 是 5 的父结点

C. 这是一棵二叉搜索树　D. 这是一棵完全二叉树

7．将森林 F 转换为对应的二叉树 T，F 中叶结点的个数为（　）。

A. T 中叶结点个数　B. T 中度为 1 的结点个数

C. T 中左子指针为空的结点个数　D. T 中右子指针为空的结点个数

8．一棵完全二叉树有 1000 个结点，则有　　个叶子结点，　　个度为 2 的结点，　　个结点具有非空左子树，　　个结点只有非空右子树。

二、辨析与简答

1．对待匹配文本 t 和模式 p，求 p 优化后的 next 数组，并画出 KMP 匹配过程。

`t = "aabaabaabaac"`，`p = "aabaac"`。

> 原图标题字符串较模糊；此处据同页逐字符表格及匹配图恢复为上述文本与模式。

2．序列 {23,17,14,6,13,10,1,5,8,12} 是否为最大值堆？若是，说明理由；否则严格按筛选法建堆过程调整为最大值堆，并画出逐步过程。

3、4．来源缺题：给定目录未提供这两题的图片，保留原编号，不补造题目。

5．使用重量权衡合并规则与路径压缩，依次合并以下 15 个等价对。初始元素 0 至 15 各自为一个集合。小树并入大树；同规模时，根值较大者并入根值较小者。写出所有操作完成后的父结点下标数组（根的父指针指向自己）。

(0,2)、(1,2)、(3,4)、(3,1)、(3,5)、(9,11)、(12,14)、(3,9)、(4,14)、(6,7)、(8,10)、(8,7)、(7,0)、(10,15)、(10,13)。

三、算法填空

1．对于全由小写字母组成的字符串，采用 `x[φ]` 表示字符串 φ 重复 x 次，φ 可为嵌套压缩串，$0 < x \le 300$。输入合法，解码后长度不超过 100000。补全解码程序。例如输入 `3[a2[c]]`，输出 `accaccacc`。

以下为结构性复原：原图仅存程序声明、结尾及四个空的答案，缺少中间代码。为便于练习，按栈方法补齐程序骨架；不视为原卷逐字代码。

```
#include <bits/stdc++.h>
using namespace std;
const int N = 100005;
char s[N], stk[N], t[N];
int top = 0;
bool isdigit2(char c) {
    return /* (1) */;
}
int main() {
    scanf("%s", s);
    for (int i = 0; s[i]; ++i) {
        if (s[i] == ']') {
            int len = 0;
            while (stk[top] != '[') {
                t[++len] = /* (2) */;
                --top;
            }
            --top;
            int digit_pos = top;
            while (digit_pos >= 1 && isdigit2(stk[digit_pos]))
                --digit_pos;
            int num = 0;
            while (isdigit2(stk[digit_pos + 1])) {
                /* (4) */;
                ++digit_pos;
            }
            top = digit_pos;
            while (top >= 1 && isdigit2(stk[top])) --top;
            for (int j = 1; j <= num; ++j)
                for (int k = len; k >= 1; --k) {
                    ++top;
                    /* (3) */;
                }
        } else stk[++top] = s[i];
    }
    for (int i = 1; i <= top; ++i) printf("%c", stk[i]);
    printf("\n");
}
```

2．给定二叉搜索树，补全程序，使用普通遍历方法寻找第 k 大的结点。所有 visited 初始为 false，k 合法，parent 指向父结点。

```
class Node {
public:
    int val;
    Node *left, *right, *parent;
    bool visited;
};
Node* kth_number_non_recursive(int k, Node* root) {
    while (root != NULL) {
        while (/* (5) */) {
            /* (6) */;
        }
        if (!root->visited) {
            /* (7) */;
            if (k == 0) break;
            root->visited = true;
        }
        if (/* (8) */) {
            /* (9) */;
        } else {
            root = root->parent;
        }
    }
    return root;
}
```

四、算法设计与实现

1．两个单链表 A、B 在某结点开始相交，整个结构无环，头地址分别为 headA、headB。设计算法找出并返回相交的起始结点，给出算法描述及伪代码。要求时间复杂度 $O(\text{len}(A)+\text{len}(B))$，额外空间 $O(1)$。

```
A: a1 → a2 ──────┐
                 c1 → c2 → c3
B: b1 → b2 → b3 ┘
```

2．长度为 n 的字符串 S，其长度为 i 的前缀为 S[0..i-1]。请设计尽量高效的算法，求所有前缀中“前缀长度 × 该前缀在 S 中出现次数”的最大值。允许重叠出现。描述算法思想，并分析时间与空间复杂度。

例：S = cdcdcdc，长度 1 至 7 的前缀出现次数依次为 4、3、3、2、2、1、1，乘积依次为 4、6、9、8、10、6、7，答案为 10。